TUGAS UNGUIDED
PEMROGRAMAN PERANGKAT BERGERAK

MODUL 1

RUNNING MODUL



Disusun Oleh :

Izzan Maula Rifqi 103122430009

Kelas S1SE-08-02

Asisten Praktikum :

Abu Abdirrahman Humaid Al-Atsary
Aflah Rizkyadhafin Nurfikri

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs.

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

TUGAS UNGUIDED

A. SOAL

TUGAS:

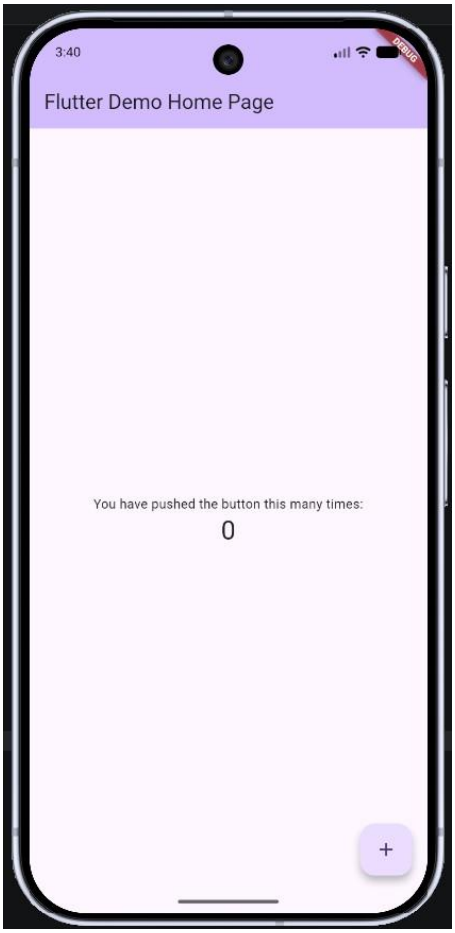
1. **Membuat Project Flutter**
   Buatlah sebuah project Flutter baru dengan nama **MOD1_NIM** menggunakan **Visual Studio Code**. Dokumentasikan / *screenshot* setiap langkah pengerjaan mulai dari membuat *project* hingga berhasil menjalankan *program*-nya.

2. **Pastikan** project **dapat dijalankan** pada emulator Android.

KETERANGAN PENGUMPULAN:

- Kumpulkan **file PDF** yang berisi *screenshot* tampilan langkah pengerjaan beserta dengan hasilnya, sertakan juga **code**-nya.
- Kumpulkan **file ZIP** yang berisi folder **lib** dari **project** yang telah dibuat.

Contoh Output:



B. JAWABAN

Source code

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        // This is the theme of your application.
        //
        // TRY THIS: Try running your application with "flutter run". You'll
see
        // the application has a purple toolbar. Then, without quitting the
app,
        // try changing the seedColor in the colorScheme below to Colors.green
        // and then invoke "hot reload" (save your changes or press the "hot
        // reload" button in a Flutter-supported IDE, or press "r" if you used
        // the command line to start the app).
        //
        // Notice that the counter didn't reset back to zero; the application
        // state is not lost during the reload. To reset the state, use hot
        // restart instead.
        //
        // This works for code too, not just values: Most code changes can be
        // tested with just a hot reload.
        colorScheme: .fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  // This widget is the home page of your application. It is stateful, meaning
  // that it has a State object (defined below) that contains fields that
affect
  // how it looks.

  // This class is the configuration for the state. It holds the values (in
this
  // case the title) provided by the parent (in this case the App widget) and
  // used by the build method of the State. Fields in a Widget subclass are
  // always marked "final".
```

```
  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

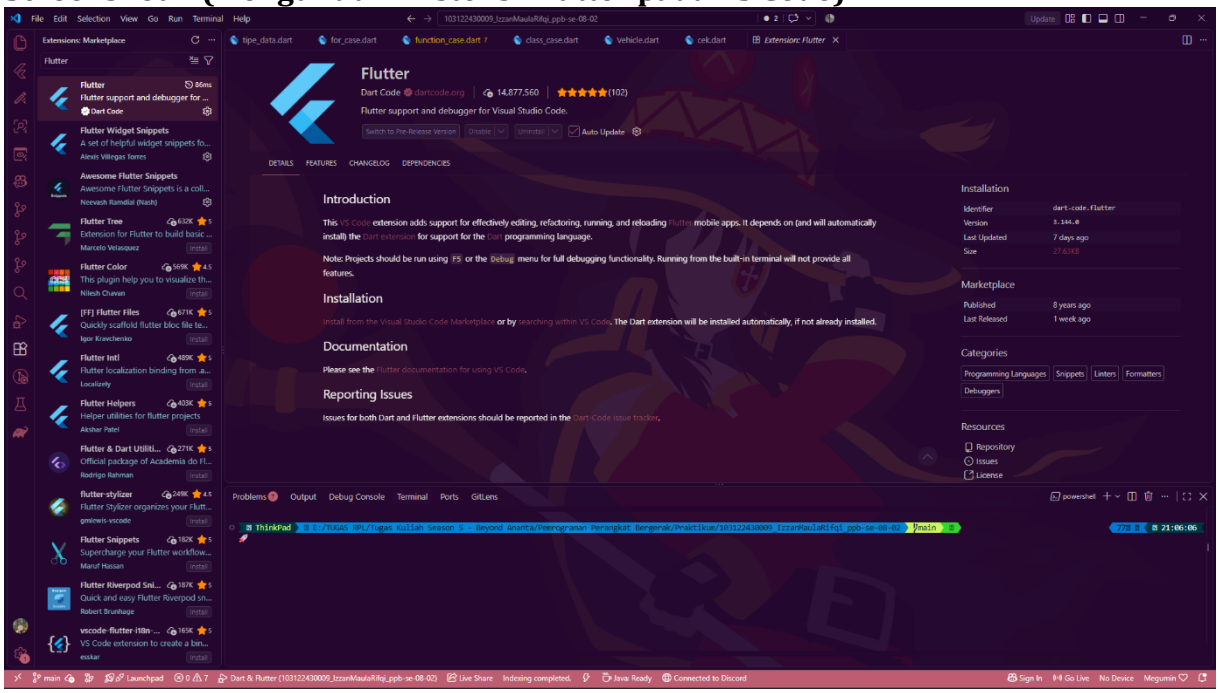
  void _incrementCounter() {
    setState(() {
      // This call to setState tells the Flutter framework that something has
      // changed in this State, which causes it to rerun the build method
below
      // so that the display can reflect the updated values. If we changed
      // _counter without calling setState(), then the build method would not
be
      // called again, and so nothing would appear to happen.
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    // This method is rerun every time setState is called, for instance as
done
    // by the _incrementCounter method above.
    //
    // The Flutter framework has been optimized to make rerunning build
methods
    // fast, so that you can just rebuild anything that needs updating rather
    // than having to individually change instances of widgets.
    return Scaffold(
      appBar: AppBar(
        // TRY THIS: Try changing the color here to a specific color (to
        // Colors.amber, perhaps?) and trigger a hot reload to see the AppBar
        // change color while the other colors stay the same.
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        // Here we take the value from the MyHomePage object that was created
by
        // the App.build method, and use it to set our appbar title.
        title: Text(widget.title),
      ),
      body: Center(
        // Center is a layout widget. It takes a single child and positions it
        // in the middle of the parent.
        child: Column(
          // Column is also a layout widget. It takes a list of children and
          // arranges them vertically. By default, it sizes itself to fit its
          // children horizontally, and tries to be as tall as its parent.
          //
          // Column has various properties to control how it sizes itself and
          // how it positions its children. Here we use mainAxisAlignment to
          // center the children vertically; the main axis here is the
vertical
          // axis because Columns are vertical (the cross axis would be
```

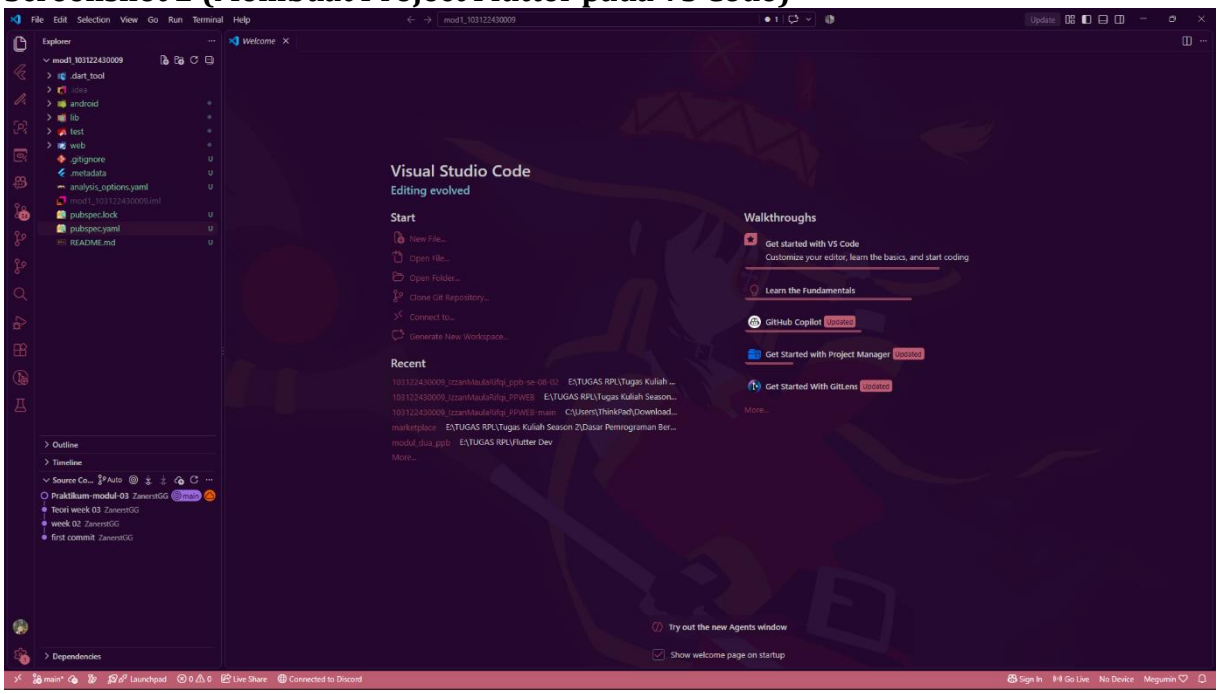
```
          // horizontal).
          //
          // TRY THIS: Invoke "debug painting" (choose the "Toggle Debug
Paint"
          // action in the IDE, or press "p" in the console), to see the
          // wireframe for each widget.
          mainAxisAlignment: .center,
          children: [
            const Text('You have pushed the button this many times:'),
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Screenshot Output

Screenshot 1 (Mengunduh Ekstensi Flutter pada VS Code)



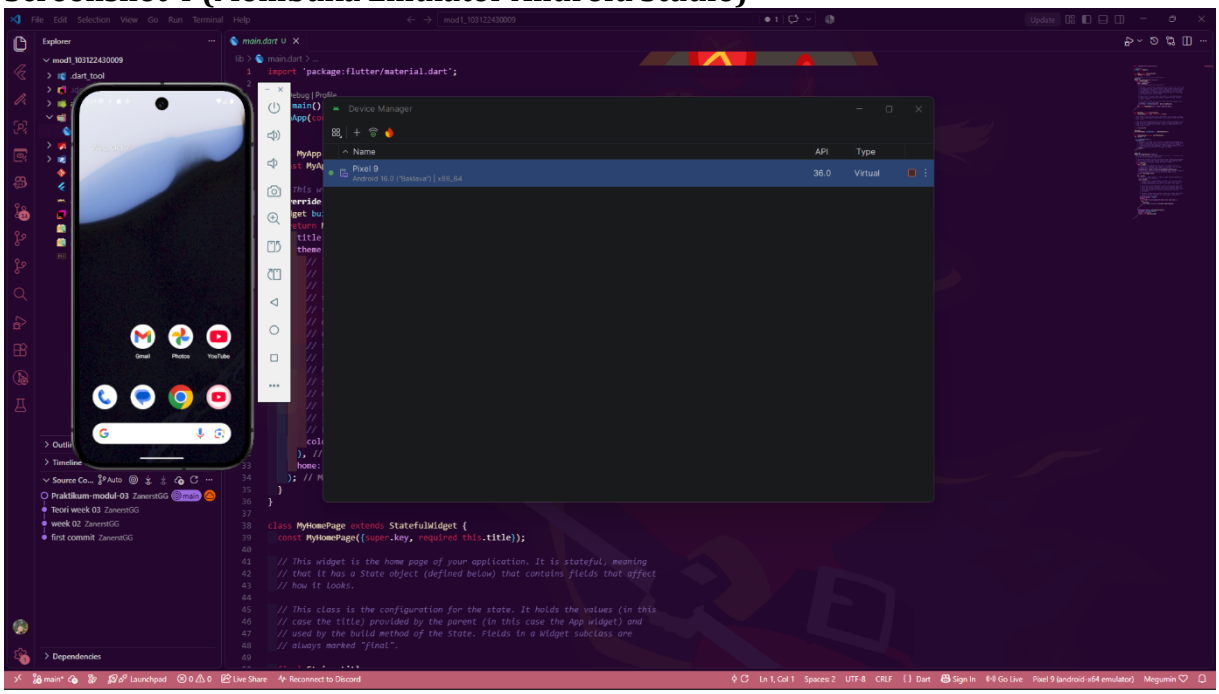
Screenshot 2 (Membuat Project Flutter pada VS Code)



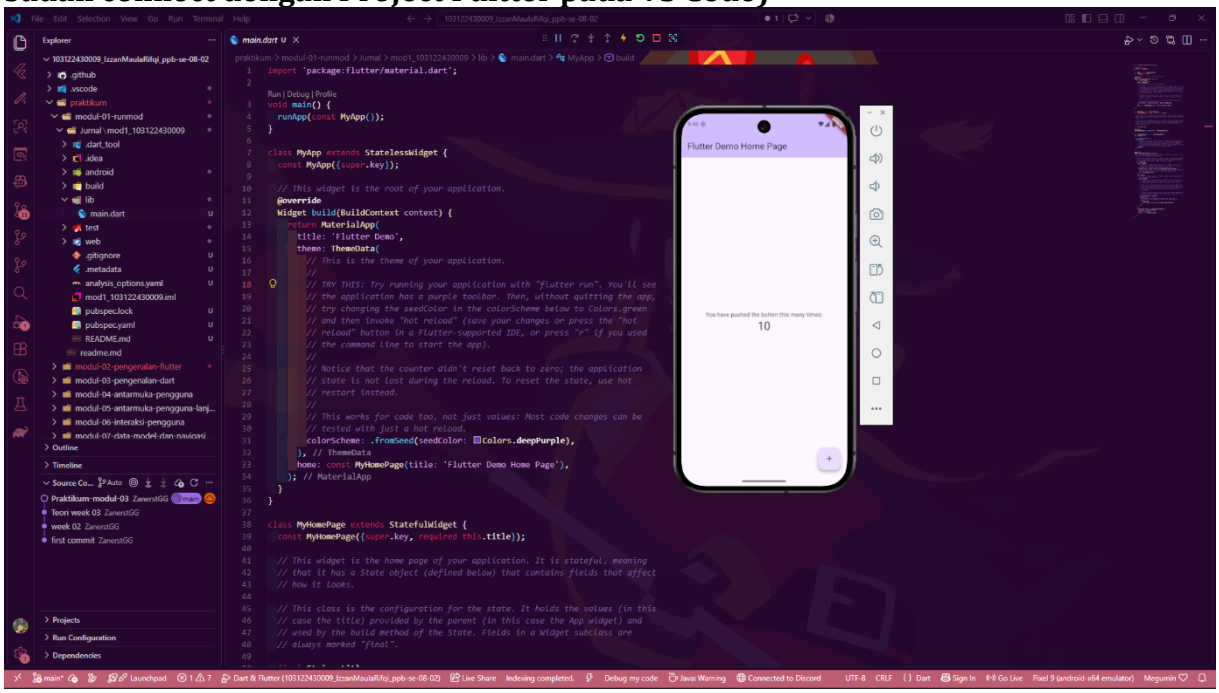
Screenshot 3 (Membuka main.dart)



Screenshot 4 (Membuka Emulator Android Studio)



Screenshot 5 (Run Debugging pada main.dart dan hasilnya Emulator Android sudah connect dengan Project Fultter pada VS Code)



Deskripsi Program

Program yang dibuat adalah aplikasi sederhana menggunakan Flutter dengan tampilan Flutter Demo Home Page. Aplikasi ini memiliki sebuah angka penghitung (counter) dan tombol tambah (+) yang digunakan untuk menaikkan angka tersebut. Algoritma yang digunakan cukup sederhana, yaitu nilai counter dimulai dari 0, lalu setiap tombol tambah ditekan, nilainya akan bertambah 1. Program menggunakan fungsi setState() agar perubahan nilai counter langsung ditampilkan pada layar. Setelah project dijalankan melalui Visual Studio Code, aplikasi akan muncul di emulator Android dan pengguna bisa menekan tombol tambah untuk mencoba fitur tersebut. Output yang dihasilkan adalah tampilan aplikasi dengan judul Flutter Demo Home Page dan angka counter yang berubah sesuai jumlah tombol yang ditekan.